CNENGR 116
Fundamentals of Digital Electronics

Lecture 9 – Counters and Registers

Dr Zhongming Zhang

Zhongming.zhang@lancaster.ac.uk

School of Engineering

Lancaster University

Objectives

To:

- Briefly revise the concept of State Machines
- Introduce counters (synchronous and asynchronous)

Finite State Machine Concept

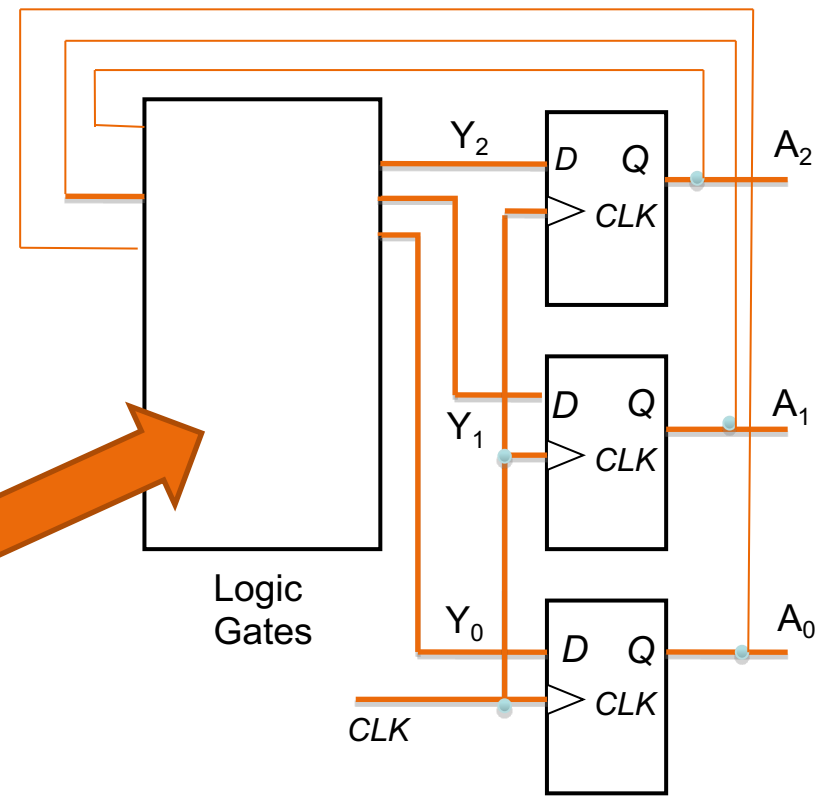
- Generates a change in response to a clock pulse based on the current state of the system and in most cases a set of input conditions
  - Example – a counter
    - No inputs
    - Transitions in response to a clock
    - Cyclic

Current State A_2, A_1, A_0	Next State Y_2, Y_1, Y_0
000	001
001	010
010	011
011	100
100	101
101	110
110	111
111	000

3-bit Counter - Design

Current State A_2, A_1, A_0	Next State Y_2, Y_1, Y_0
000	001
001	010
010	011
011	100
100	101
101	110
110	111
111	000

$Y_2=f(A_2, A_1, A_0)$
$Y_1=f(A_2, A_1, A_0)$
$Y_0=f(A_2, A_1, A_0)$



3-bit counter

Current State A_2, A_1, A_0	Next State Y_2, Y_1, Y_0
000	001
001	010
010	011
011	100
100	101
101	110
110	111
111	000

$$\boldsymbol{Y_0 = \overline{A_0}}$$

$$Y_1 = \overline{A_2}\,\overline{A_1}A_0 + \overline{A_2}A_1\overline{A_0} + A_2\overline{A_1}A_0 + A_2A_1\overline{A_0}$$
$$Y_1 = A_1\overline{A_0}(\overline{A_2} + A_2) + \overline{A_1}A_0(\overline{A_2} + A_2)$$
$$\boldsymbol{Y_1 = A_1 \oplus A_0}$$

$$\boldsymbol{Y_2 = A_2\overline{A_1} + A_2\overline{A_0} + \overline{A_2}A_1A_0}$$

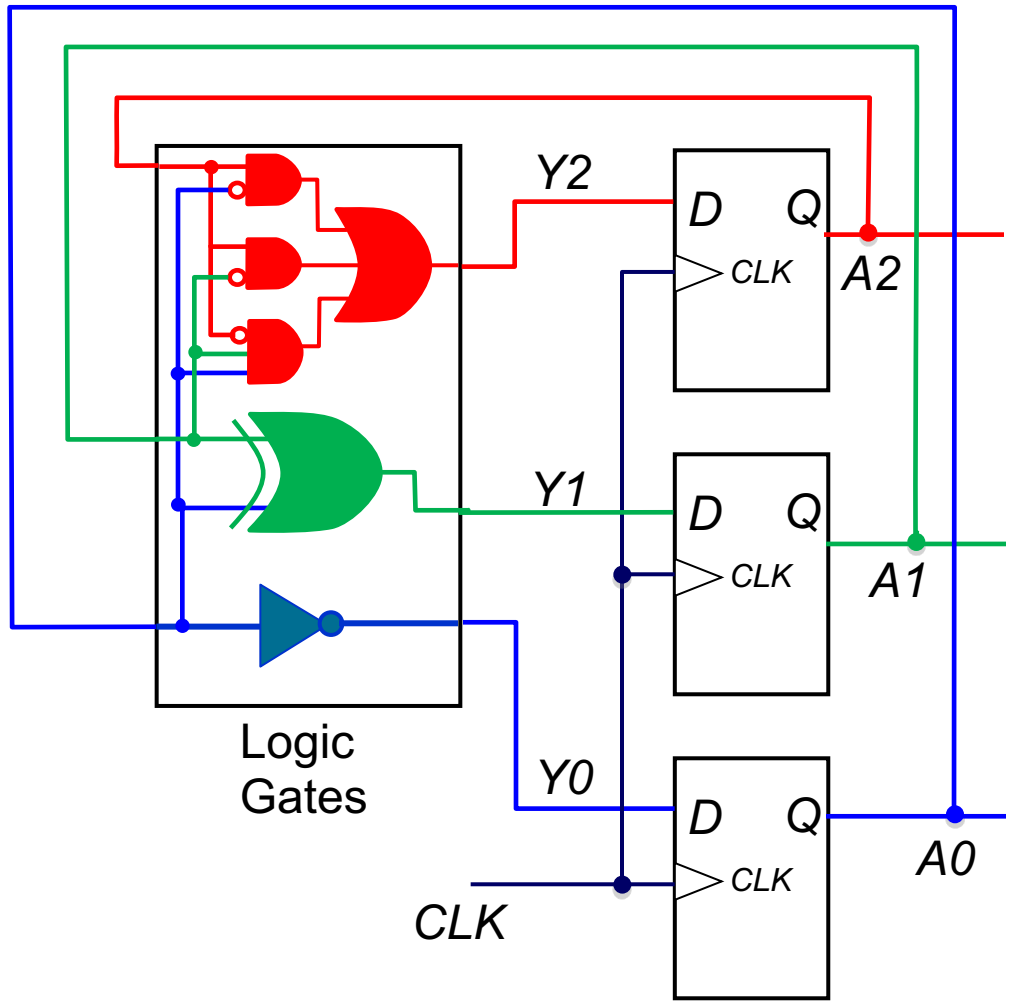
Calculate the outputs from the logic required to define the next state

FSM 3-bit counter

$$\boldsymbol{Y_0 = \overline{A_0}}$$

$$Y_1 = \overline{A_2}\,\overline{A_1}A_0 + \overline{A_2}A_1\overline{A_0} + A_2\overline{A_1}A_0 + A_2A_1\overline{A_0}$$
$$Y_1 = A_1\overline{A_0}(\overline{A_2} + A_2) + \overline{A_1}A_0(\overline{A_2} + A_2)$$
$$\boldsymbol{Y_1 = A_1 \oplus A_0}$$

$$\boldsymbol{Y_2 = A_2\overline{A_1} + A_2\overline{A_0} + \overline{A_2}A_1A_0}$$



This design will now count on every pulse of the clock

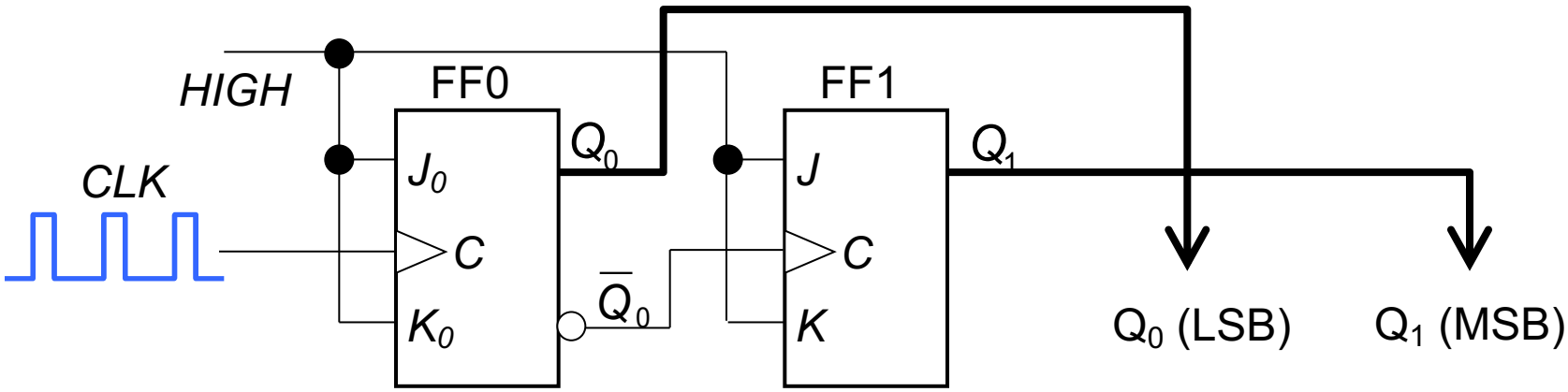
Counters

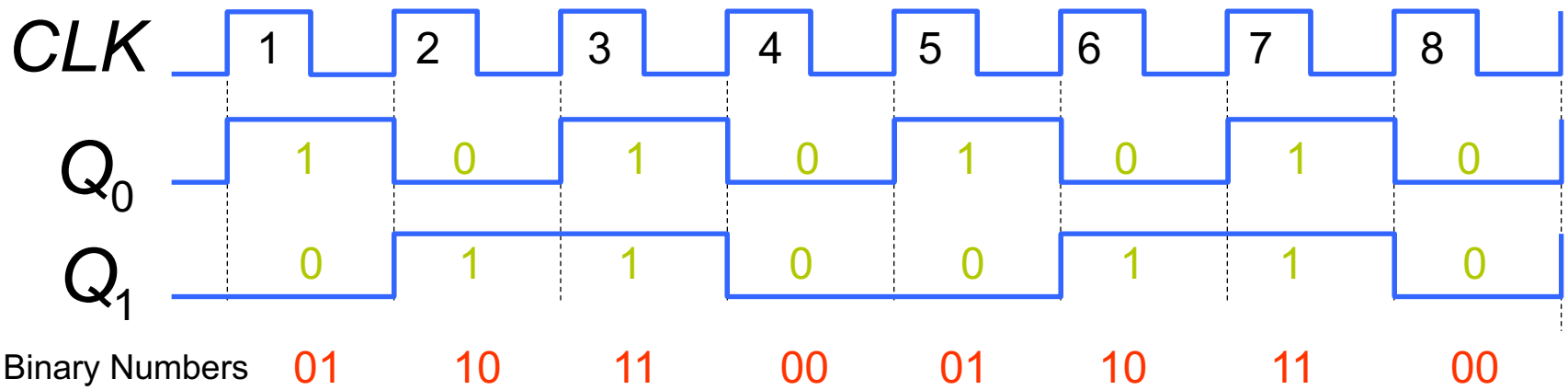
- An **asynchronous** counter is one in which the flip-flops within the counter do NOT change states at exactly the same time because they do not have a common clock.
- A **synchronous** counter is one in which all the flip-flops in the counter are clocked at the same time by a common clock.
- The **modulus** of a counter is the number of unique states through which the counter will sequence. The maximum possible number of states is 2^n, where n is the number of flip-flops in the counter.

We do not have to use the State Machine concept and D-type's to build counters. We can make the logic simpler if we wish by using J-K flip-flops implemented in toggle mode

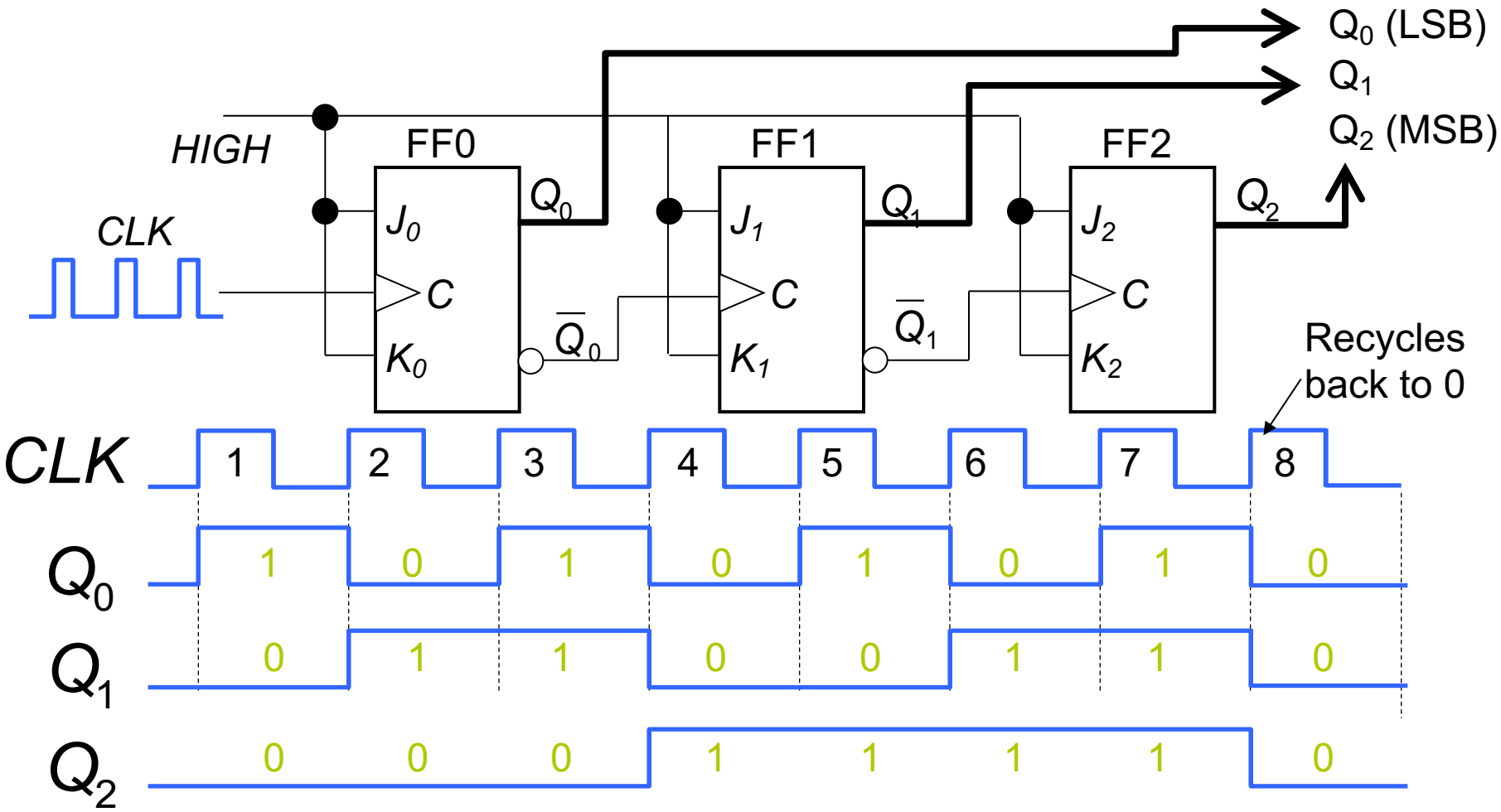
2-bit Asynchronous Binary Counter

- One of the applications of a flip-flop is dividing the frequency of a periodic waveform.





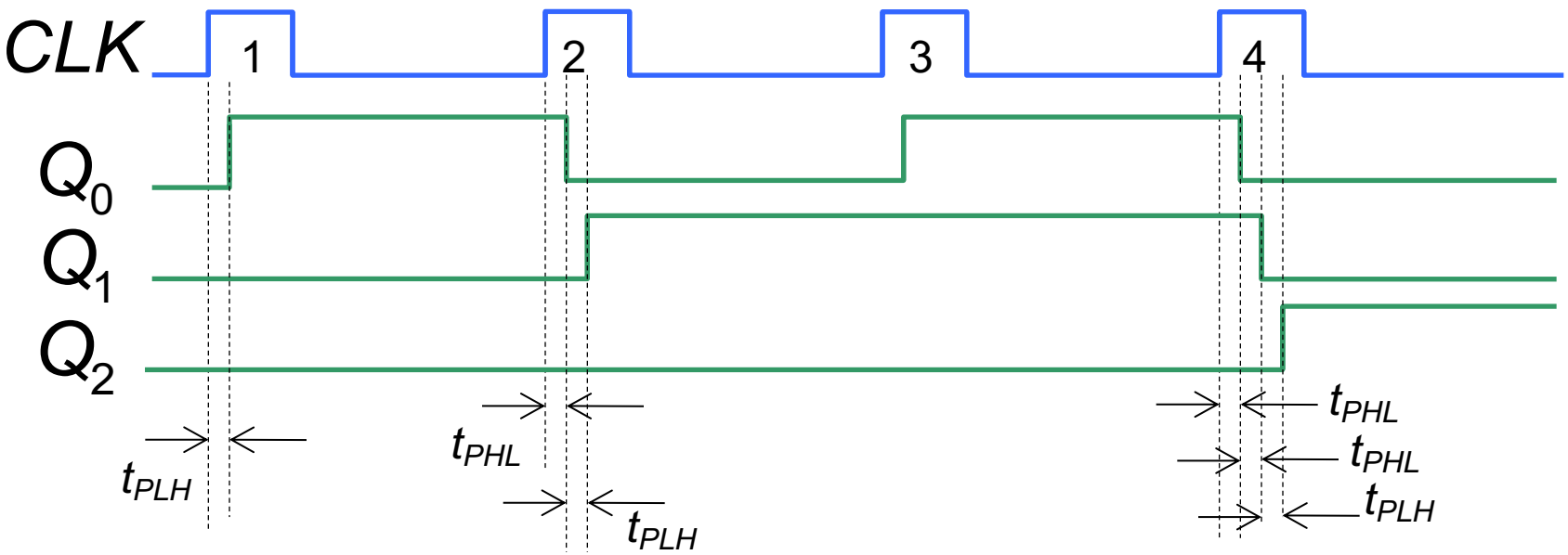
3-bit Asynchronous Binary Counter



Binary Numbers	001	010	011	100	101	110	111	000

Propagation delay

- Asynchronous counters are also known as ripple counters since the effect of the input clock pulse is first “felt” by FF0 than by FF1, then by FF2
- Realistic flip-flops have a delay time for passing on the signal and therefore would introduce a time shift which would increase the further down the line we go.



A Quick Exercise

- Noting the given table to count the binary numbers how could an asynchronous up-counter be modified to change it to a down-counter?

Q4	Q3	Q2	Q1
1	1	1	1
1	1	1	0
1	1	0	1
1	1	0	0
1	0	1	1
1	0	1	0
1	0	0	1
0	1	1	1
0	1	1	0
0	1	0	1
0	1	0	0
0	0	1	1

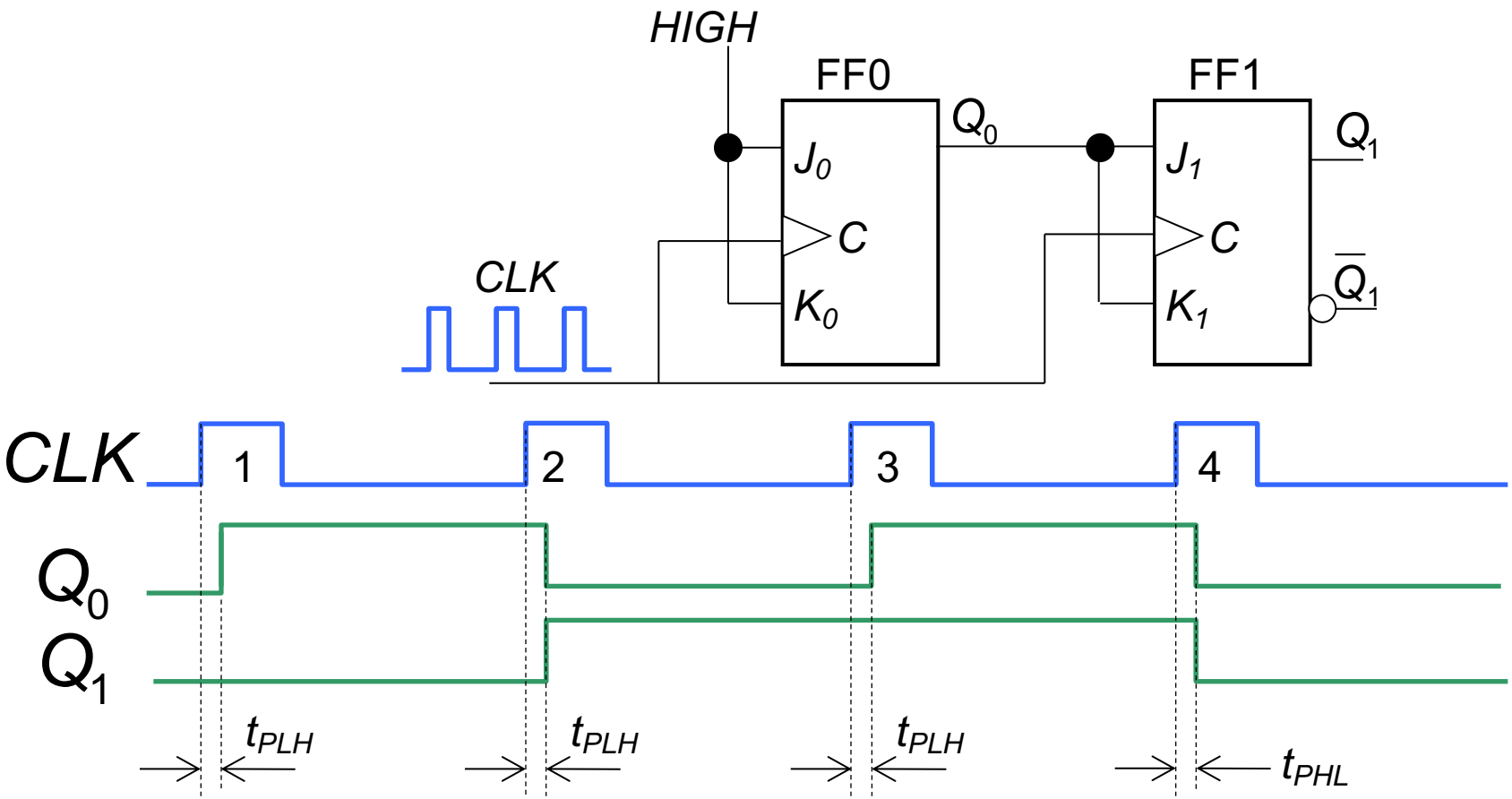
NOT AVAILABLE IN PREVIEW

A Quick Exercise

NOT AVAILABLE IN PREVIEW

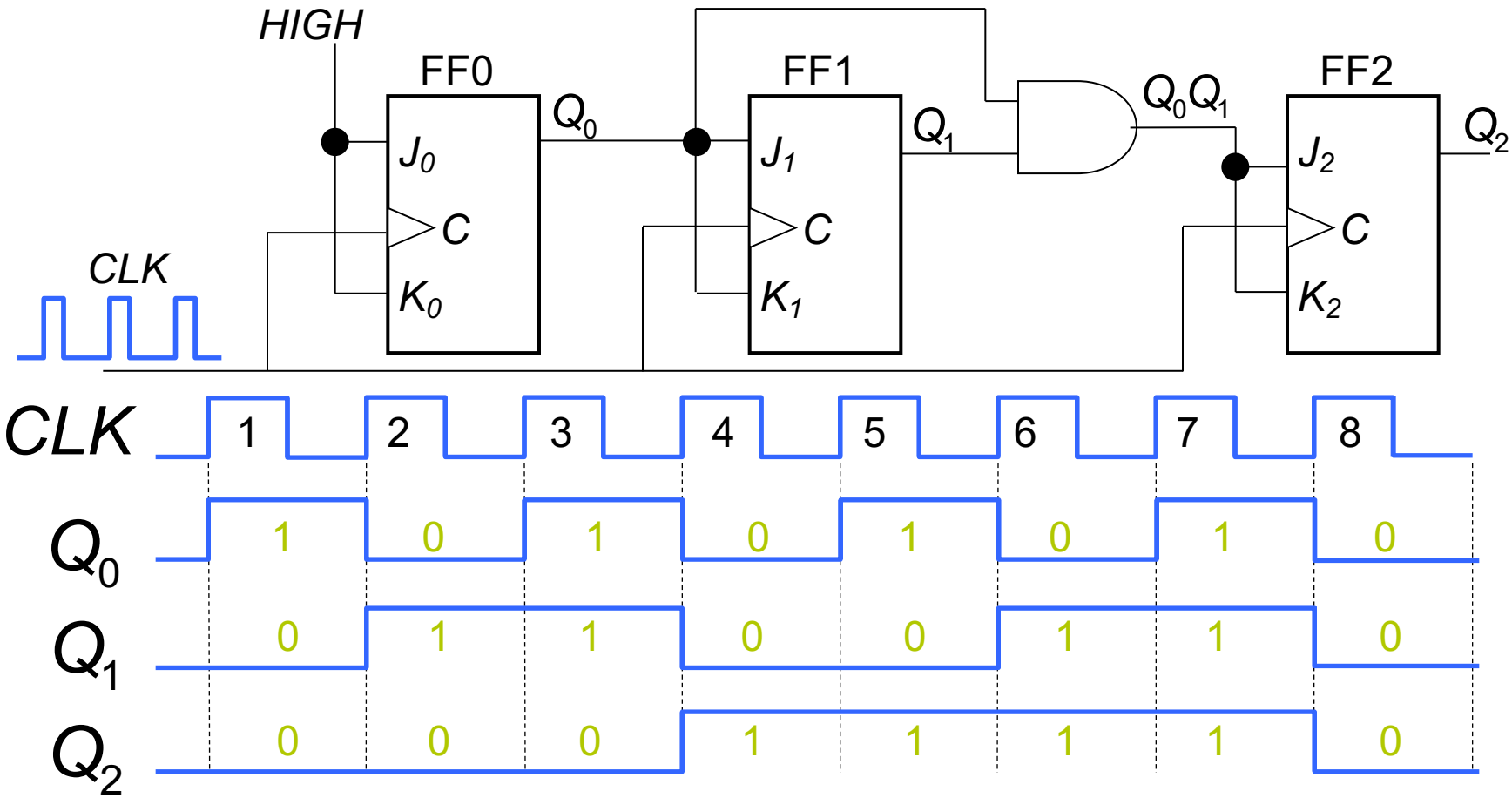
2-bit Synchronous Binary Counter

- This time we have a common clock line. The same delay is found in all signals making them simultaneous (or synchronous)



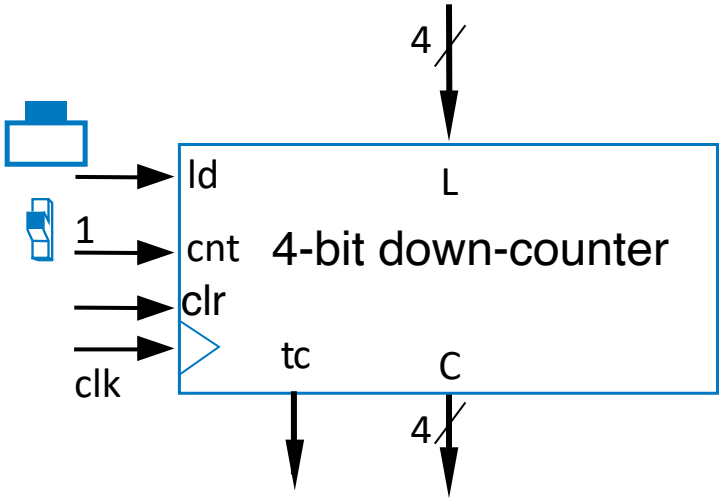
3-bit Synchronous Binary Counter

- This time we have a common clock line. The same delay is found in all signals making them simultaneous (or synchronous)



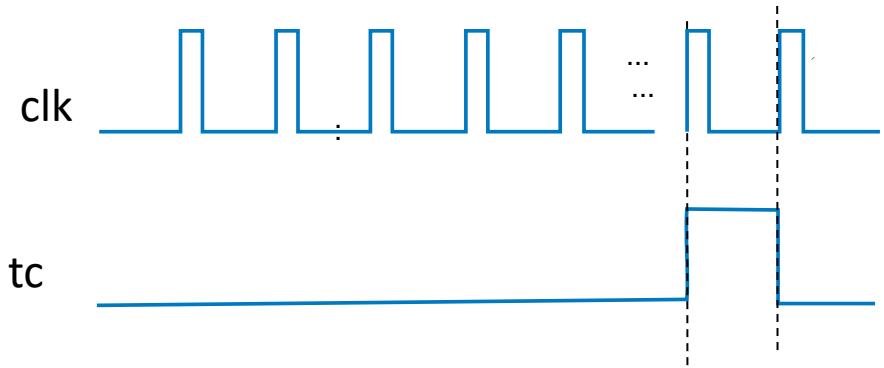
Common as a Logic Component

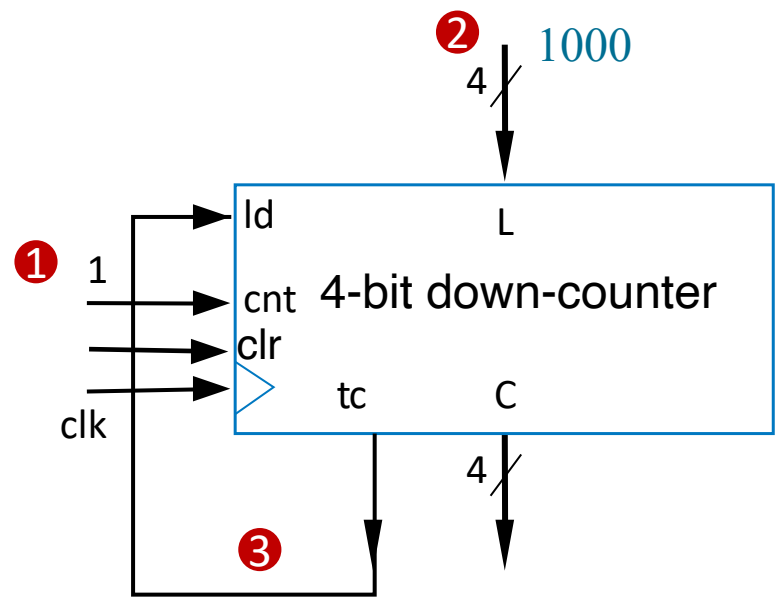
- **cnt**: Synchronous enable to start counting
- **ld**: Synchronous parallel load to initialize counter with a specific value **L**
- **clr:** Synchronous clear to start counting from zero
- **tc:** terminal count to show count has reached its final state
- **C**: output of counter



Common Use of Counter

- **Creating Pulses at Specific Multiples of Clock**
- **Example:** Configure the 4-bit down-counter below to generate a pulse on every 9th clock cycle

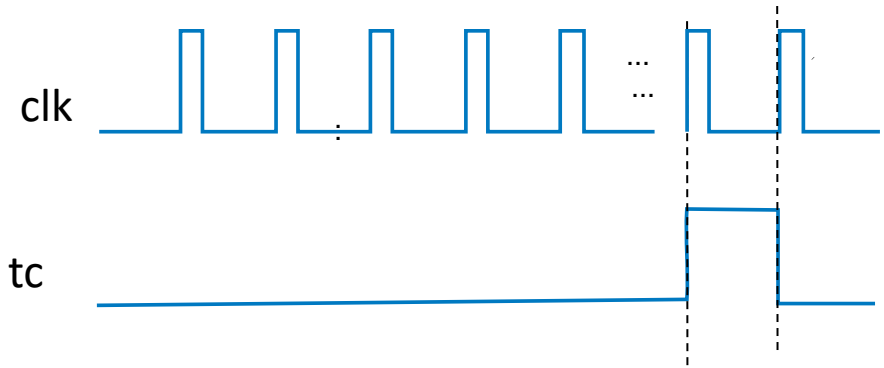


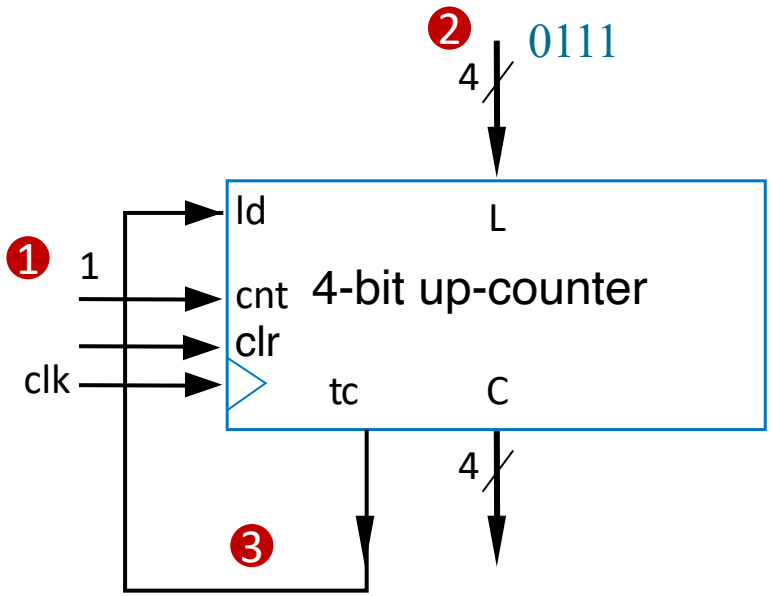


- Set parallel load input to 9-1=8 (1000)
- Use terminal count to reload

Common Use of Counter

- **Creating Pulses at Specific Multiples of Clock**
- **Example:** Configure the 4-bit up-counter below to generate a pulse on every 9th clock cycles





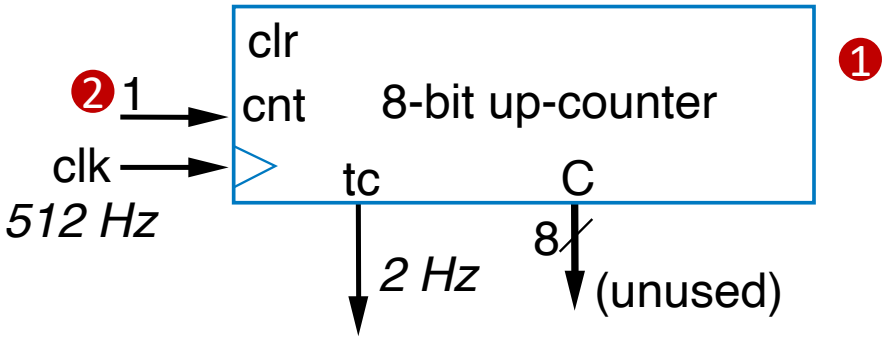
- Set parallel load input to 15-8=7 (0111)
- Use terminal count to reload

Common Use of Counter

- **Frequency Division by Power of 2**
- **Example:** Suppose we have an oscillator with clock frequency 512Hz. Using an up-counter generate a 2 Hz clock pulse.
  - We want to divide the clock by 256
  - For the design we need to use an 8-bit up-counter and use tc output as the pulse generator
  - Counts from 0 to 255

(256 counts), so pulses tc

every 256 cycles

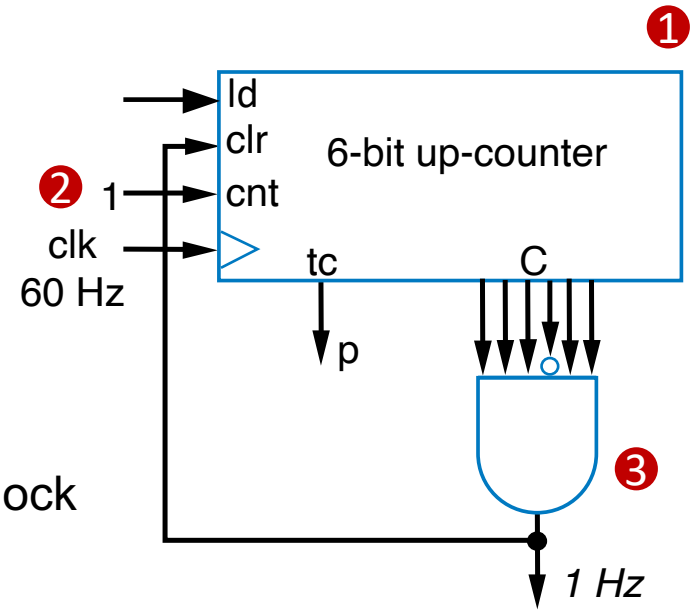


Common Use of Counter

- **Clock Division by something other than the Power of 2**

- **Example:** The frequency of the electricity signal delivered to houses is 60 Hz. You are asked to design an up-counter to convert that to 1 Hz signal.

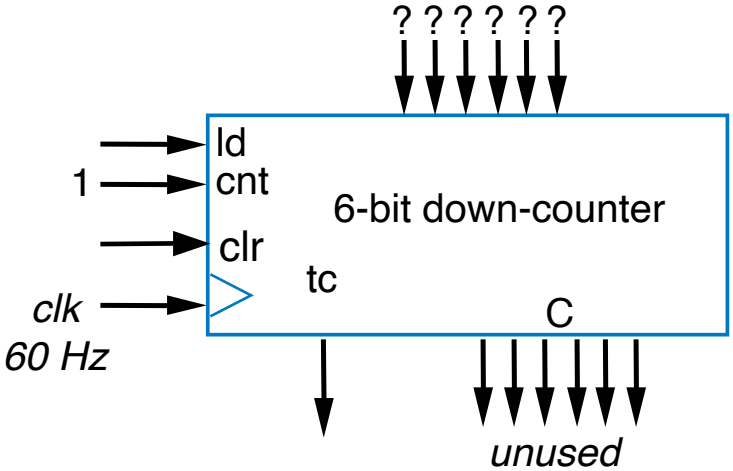
  - We need to use a 6-bit up-counter
  - Detected count of 60-1=59 (111011)
  - Clears counter on the next rising edge
  - tc thus pulses every 60 cycles



1 Hz Pulse Generator from 60 Hz Clock

A Quick Exercise

- Try to convert a 60 Hz clock signal to a 1 Hz signal using a 6-bit down-counter.
  - Using a 6-bit down counter
  - Set parallel load input to?
  - Using terminal count to reload
  - tc thus pulses every 60 cycles



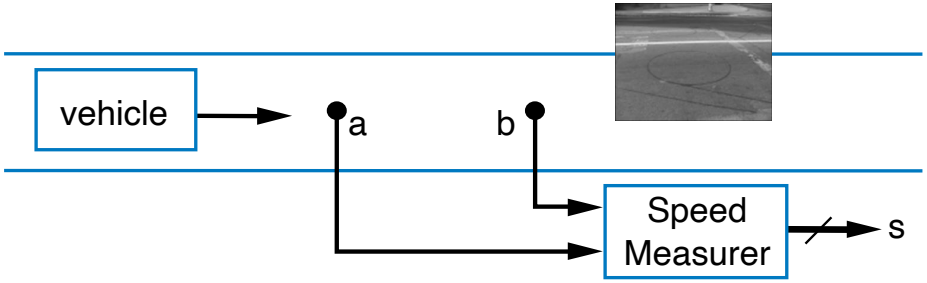
A Quick Exercise

Solution:

NOT AVAILABLE IN PREVIEW

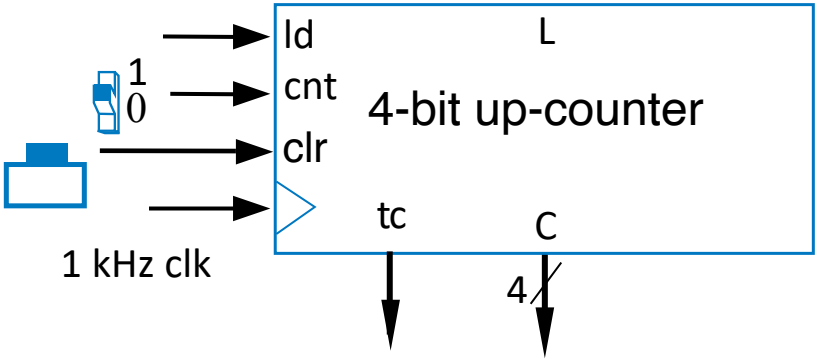
Counter as Timer

- **Measuring Time Between Events**
- **Example:** You are asked to design an electronic device to measure the speed of a car on a highway between two points a and b.
  - Two sensors “a” and “b” in the road to detect the car
  - Initially clear the counter to 0
  - Set cnt=1 for the 1st event
  - Set cnt=0 for the 2nd event



Counter as Timer

- If clock is 1 kHz (period is 1 ms), then time is C × 0.001 sec
- If the distance between **a** and **b** is **d** meter, then vehicle speed is **d** / (C × 0.001)



Summary

- Introduced counter as a sequential logic component
- Internal design of synchronous and asynchronous counters
- Learned how to configure a counter for a specific application